

Apache Kafka

- Apache Kafka Introduction
- Apache Kafka Concepts
- Cluster Installation
- Kafka Administration
- Kafka Configurations
- Kafka Operations
- Using Kafdrop

APACHE KAFKA · FOR SYSTEM ADMINISTRATORS

Introduction to Apache Kafka

Module 1 — The Big Picture

“The circulatory system of modern data.”



THE DEFINITION

What is Apache Kafka?

A **distributed streaming platform** — that does two things:



Create real-time streams

Move data from where it is born to where it is needed, the moment it happens.



Process those streams

Compute and produce results continuously, in real time — not in overnight batches.

“Real-time” means **minutes** → **seconds** → **milliseconds**

WHY PROCESSING MUST BE CONTINUOUS

Real-time stream processing



Monitor household load

Read every meter value as it arrives.



Load > 2 kVA

A threshold is crossed.



Send SMS instantly

Alert the homeowner in seconds.



An alert an hour late is worthless.

Don't **accumulate** then process.

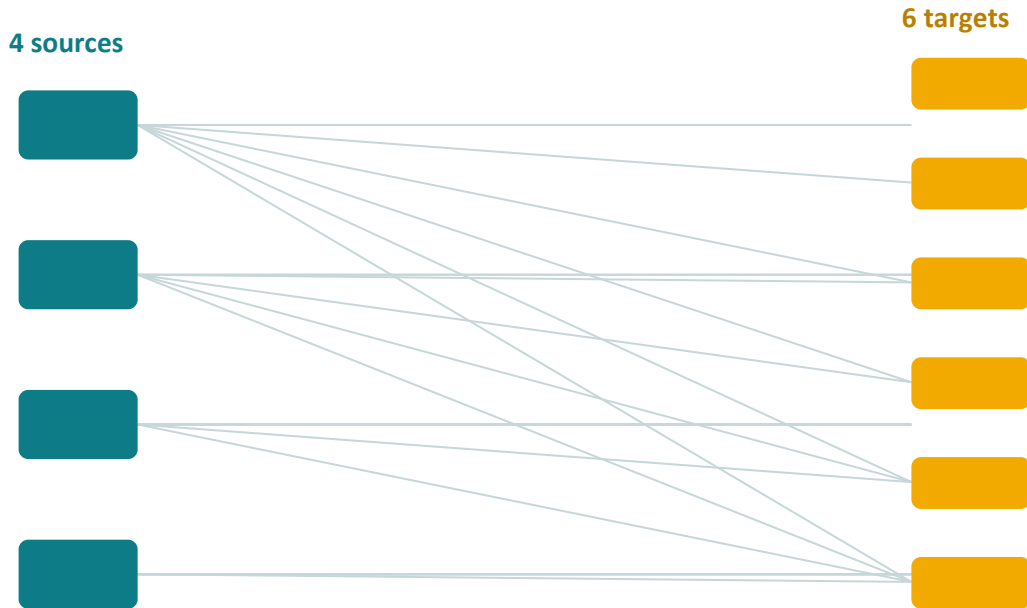
Listen continuously, act on arrival.

That is real-time stream processing.

Definition: a highly scalable, distributed platform for creating and processing streams of data in real time.

THE PROBLEM KAFKA WAS BORN TO SOLVE

Data integration: the spaghetti



$$4 \times 6 = 24 \text{ integrations}$$

Every link fights:

Protocol

TCP · HTTP · FTP · JDBC

Format

CSV · JSON · Avro · Protobuf

Schema change

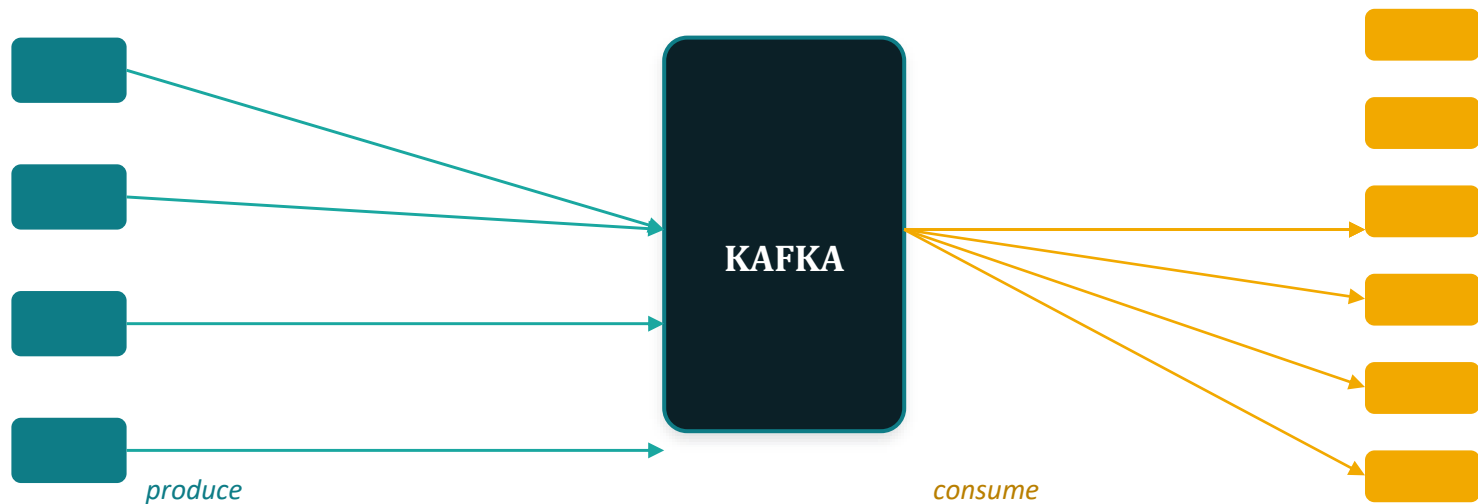
breaks downstream

Added load

on each source

THE FIX

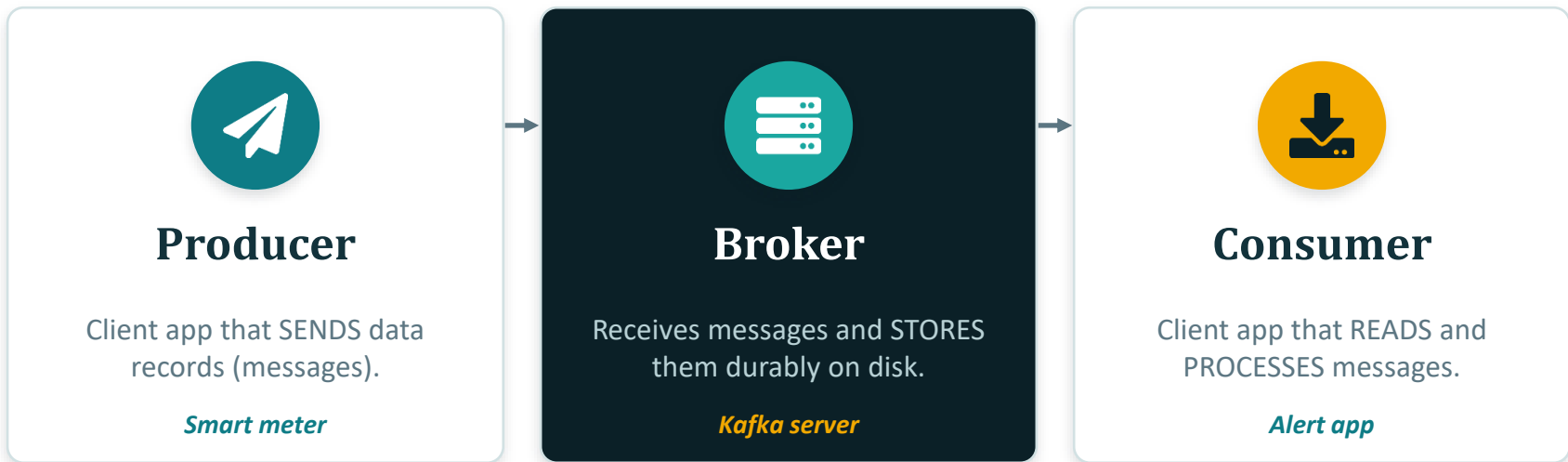
Put Kafka in the middle: decoupling



$N \times M$ spaghetti → $N + M$ clean connections · sources & targets never know each other

UNDER THE HOOD

How it works: the Pub/Sub model



The broker sits in the center as the middleman — and writes every message down, it doesn't just pass them through.

ORIGIN STORY

Where did Kafka come from?



LinkedIn

Conceived & built in-house to tame its own data-integration spaghetti.



2011

Open-sourced; later a top-level Apache project — hence Apache Kafka.



Jay Kreps

Led Kafka at LinkedIn; now co-founder & CEO of Confluent.



Confluent + others

Backed today by Confluent, IBM, Cloudera, LinkedIn & more.

Built first for data integration — the streaming-platform identity came later.

FROM INTEGRATION TOOL TO STREAMING PLATFORM

The five components

Started with 2 (broker + client API). Added 3 to become a full streaming platform.



1. Kafka Broker

The central server — the cluster.

Apache 2.0



2. Client APIs

Producer & consumer libraries.

Apache 2.0



3. Kafka Connect

Ready-made source/sink connectors — integration without code.

Apache 2.0



4. Kafka Streams

Library for real-time stream-processing apps.

Apache 2.0



5. ksqlDB (KSQL)

SQL on streams — toward a real-time database.

Confluent license

KEY STRENGTHS

Why is Kafka so good?



Fault-tolerant

Rolling upgrades & maintenance — no full outage



100s

of brokers — scale horizontally, add nodes over time



Millions

of messages per second (Twitter-scale throughput)



< 10 ms

end-to-end latency — why we call it real-time



~80%

of the Fortune 100 use Kafka · 2,000+ firms



Huge ecosystem

LinkedIn · Netflix · Uber · Airbnb · Walmart

IN PRACTICE

How Kafka is used



Messaging

A scalable message broker



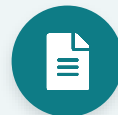
Activity tracking

Clicks, views, interactions



Metrics

Gathered from many sources



Log aggregation

Central app & server logs



Stream processing

Real-time compute
(Streams/ksqlDB)



Microservices

Decoupled, event-driven Pub/Sub



Big-data integration

Spark · Flink · Storm · Hadoop



The transport layer

Moves huge data flows reliably

KAFKA IN THE WILD

Real-world examples



Netflix

Applies recommendations in real time while you watch.



Uber

Real-time trip data → demand forecasting & surge pricing.



LinkedIn

Prevents spam & powers connection recommendations.

In all three, Kafka is purely the transport — the clever logic lives in other systems.

WHERE KAFKA FITS

The circulatory system of your data



Produced as events happen, consumed on arrival — often in milliseconds.



Producers & consumers fully decoupled — one consistent interface.



Add, remove or change producers/consumers as the business evolves.

APACHE KAFKA · FOR SYSTEM ADMINISTRATORS

Producer · Consumer · Broker · Cluster · Topic · Partition · Offset · Consumer Group



OUR MAP FOR THIS MODULE

The eight core terms



Producer

Sends messages in



Consumer

Reads messages out



Broker

One Kafka server



Cluster

A group of brokers



Topic

A named data stream



Partition

A slice of a topic



Offset

Position in a partition



Consumer Group

Consumers sharing work

Producer

An application that **sends data** to Kafka. Each piece of data is a **message**.



To Kafka, a message is just an array of bytes

Your schema, format, and meaning are the application's concern — Kafka faithfully stores and moves the bytes, nothing more.

One message per...



File → one message per line



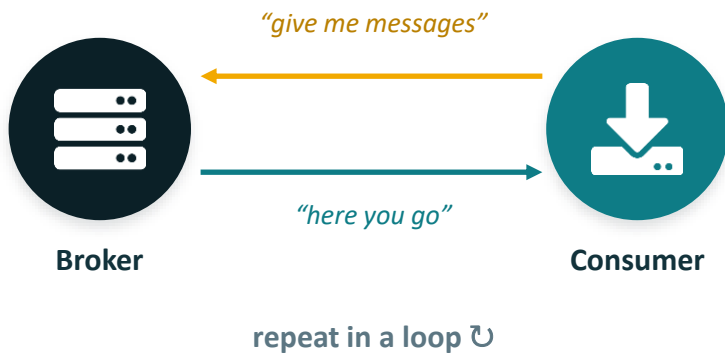
DB table → one message per row



Query result → one message per returned row

Consumer

An application that **reads data** — by **pulling** messages from the broker. Never directly from a producer.



request → receive → process → request more ...



pull model = consumer controls its own pace



what to do with the data is up to the consumer

Broker

The broker IS the Kafka server — the agent in the middle.

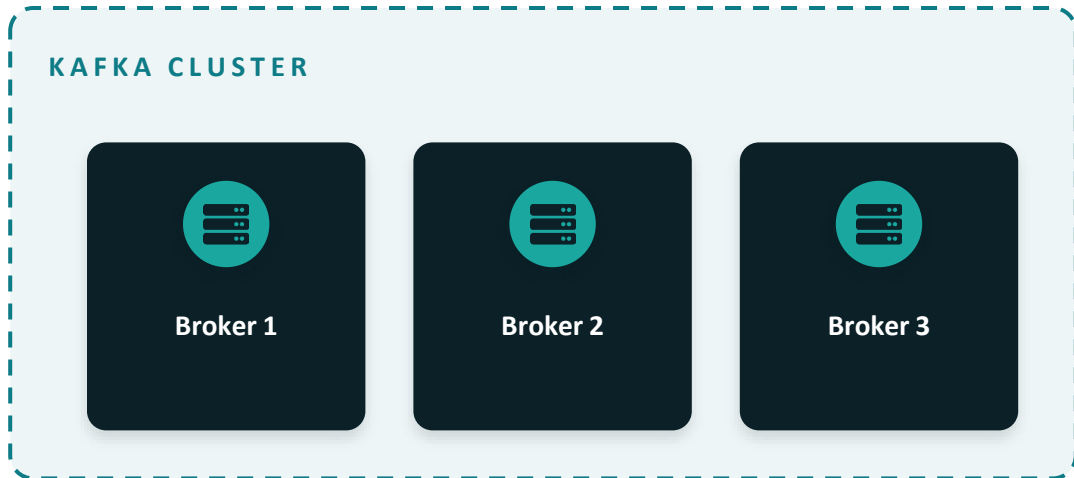


no direct producer ↔ consumer contact

One broker = one Kafka server instance. It receives, stores, and serves messages.

Cluster

A group of machines — each running one broker — working together as a distributed system.



Capacity

Spread data & traffic across machines — add a broker to grow.



Resilience

Data is spread (and replicated) so one broker dying ≠ outage.

Topic

A unique name for a **data stream**. Think of it like a **database table** — producers insert, consumers read.



current-load-topic

producer inserts ↓ consumer reads ↑ (via broker)

Design-time decision — the architect creates topics; clients refer to them by name.



current-load · every minute



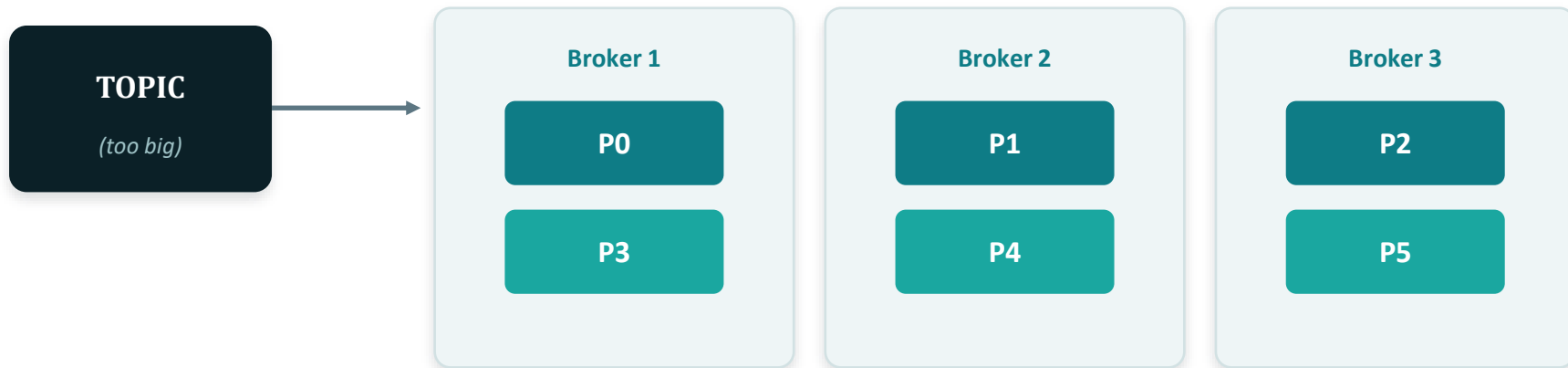
consumed-units · every hour



input-fluctuations · as they happen

Partitions — splitting the topic

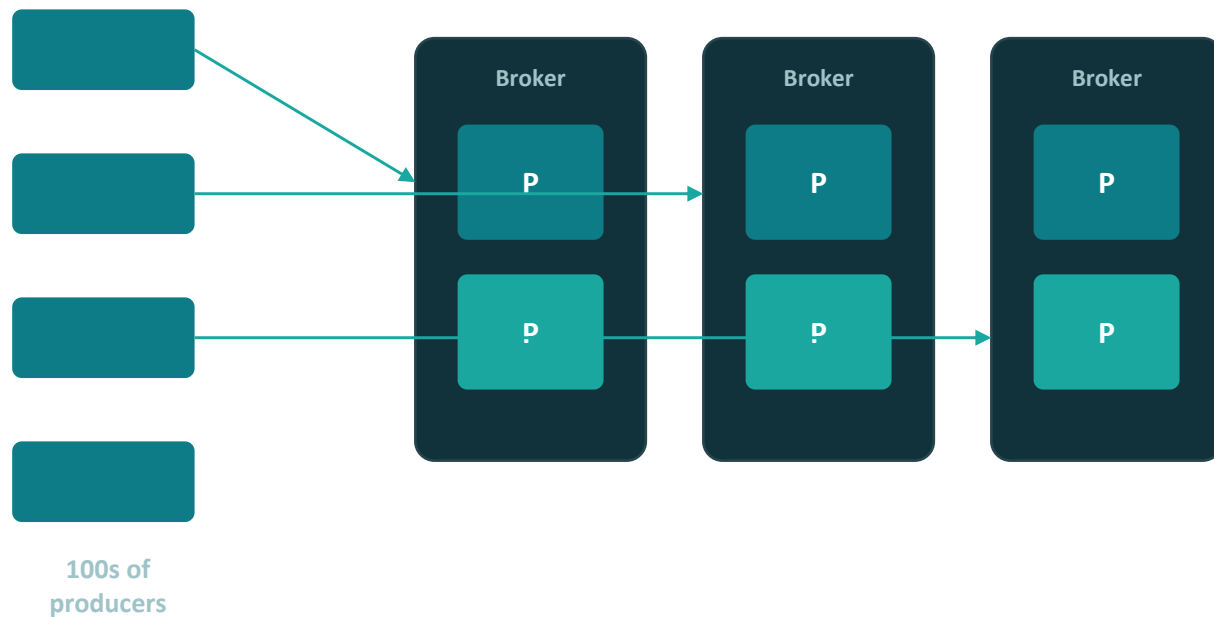
A topic can be bigger than one machine's disk. Split it into partitions and spread them across the cluster.



Partition count is YOUR design decision

A partition is the smallest unit — it lives on a single machine and can't be split further (so estimate carefully).

TERM 6 OF 8 · WHY IT MATTERS MOST



Every broker holds partitions → writes land in parallel



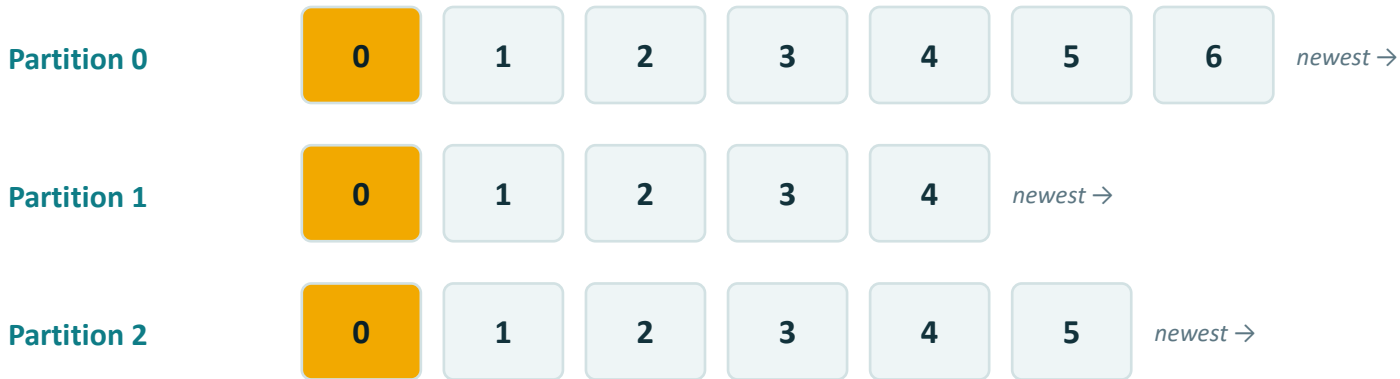
Load is shared across the cluster, not bottlenecked



Storage, write throughput AND read parallelism all scale with partitions

Offset

An immutable sequence ID for a message within a partition — assigned by the broker as it arrives.



Each partition counts from 0 independently — offsets are local; there is no global ordering across the topic.

Locate any message: **topic name + partition number + offset**

Consumer Group

Multiple consumers cooperating to share the workload — Kafka divides the partitions among them.



6 partitions, shared among the group



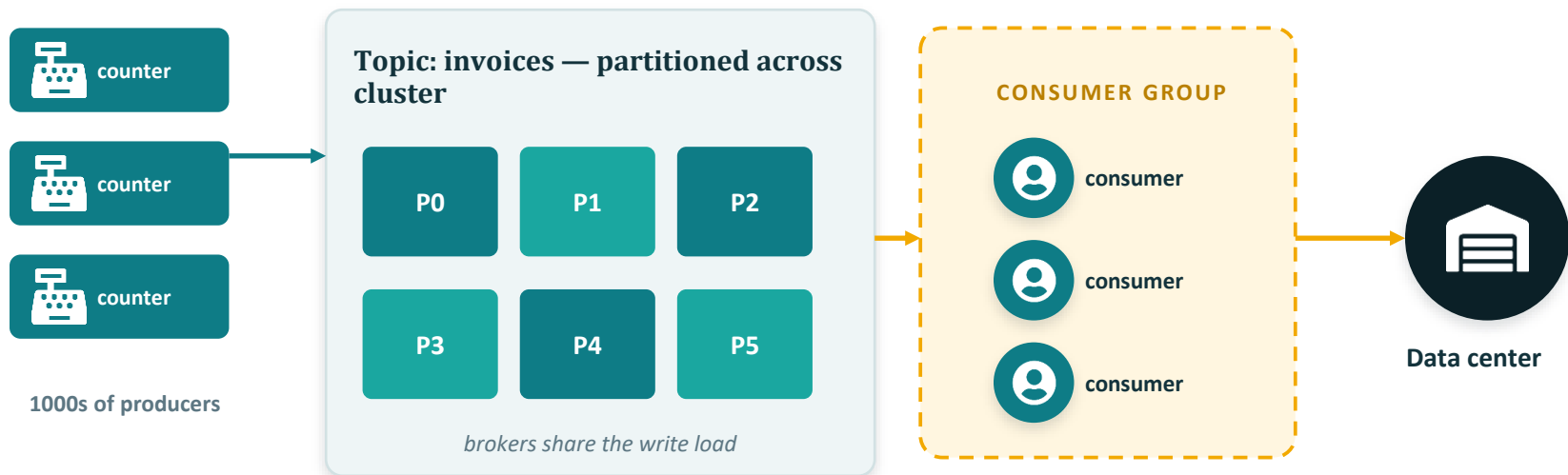
Parallelism ceiling

Max useful consumers =
number of partitions.
One consumer per partition
per group (no double-
reading).

500 part. → up to 500 consumers

PUTTING IT ALL TOGETHER

Retail example: invoices → data center



Partitions scale the write side · the consumer group scales the read side